



# Plan de Trabajo

Plugin QGIS para Descarga de Datos Catastrales



Para principiantes |



Con IA |



100% Gratuito



**Cómo usar este plan:** Cada fase contiene pasos con su **prompt exacto** para copiar y pegar en la IA (Twinny). Marca tu progreso, despliega los pasos y tacha lo completado. ¡Ve paso a paso!



## Fase 0: Configuración del Entorno



### Paso 0.1: Instalar VS Code y extensiones



**Acción:** Descargar Visual Studio Code desde `code.visualstudio.com`

**Extensiones a instalar (todas gratuitas):**

- Python (Microsoft)
- Pylance (Microsoft)
- Ruff (Charlie Marsh)
- Error Lens (Alexander)
- indent-rainbow (oderwat)
- Twinny (twinny)

### Paso 0.2: Instalar Ollama y el modelo IA



**Acción:** Descargar Ollama desde `ollama.com`

```
ollama pull deepseek-coder:6.7b-instruct-q8_0
```

**Verificar:**

```
ollama list
```

### Paso 0.3: Configurar VS Code



Archivo: `.vscode/settings.json`

```
{
  "python.languageServer": "Pylance",
  "python.analysis.typeCheckingMode": "basic",
  "editor.formatOnSave": true,
  "editor.codeActionsOnSave": {
    "source.fixAll.ruff": "explicit",
    "source.organizeImports.ruff": "explicit"
  },
  "[python]": {
    "editor.defaultFormatter": "charliermarsh.ruff"
  }
}
```

### Paso 0.4: Verificar QGIS



#### PROMPT A LA IA:

Quiero verificar que mi QGIS está listo para desarrollar plugins. Dame un comando de Python que pueda ejecutar en la consola Python de QGIS para confirmar que todo funciona. Debe mostrar "TODO OK" si no hay errores.

### Paso 0.5: Estructura del plugin





### PROMPT A LA IA:

Voy a crear mi primer plugin de QGIS. Necesito la estructura de carpetas y archivos MÍNIMA para que QGIS reconozca el plugin. El plugin se llamará "catastro\_loader". Dame lista de archivos, contenido y dónde colocarlos.



## Fase 1: Script Independiente (Fuera de QGIS) ▼

### Paso 1.1: Descargar y mostrar municipios ▼



### PROMPT A LA IA:

Escribe un script de Python INDEPENDIENTE (que NO use QGIS) que acceda al ATOM de parcelas (<https://www.catastro.hacienda.gob.es/INSPIRE/CadastralParcelas/ES.SDGC.CP.atom.xml>), extraiga todos los municipios y los muestre en consola con formato "CÓDIGO - NOMBRE". Usa requests y xml.etree.ElementTree. Incluye comentarios.

### Ejecutar:

```
python test_parcelas.py
```

## Paso 1.2: Buscar municipio y obtener URL



### PROMPT A LA IA:

```
Extiende el script. Añade
función que pida al usuario escribir un municipio, lo
busque (sin distinguir mayúsculas), muestre
coincidencias, extraiga la URL del ZIP y la imprima.
```

## Paso 1.3: Descargar y descomprimir



### PROMPT A LA IA:

```
Añade descarga del ZIP,
guardar en carpeta "descargas/", descomprimir, mostrar
archivos extraídos, identificar el archivo .gml.
```

## Paso 1.4: Validar GML



### PROMPT A LA IA:

```
Añade función que lea el GML,
muestre primeras 10 líneas, cuente etiquetas, verifique
que no está vacío y guarde resumen. Dame el script
```

COMPLETO final.

## Fase 2: Plugin Básico en QGIS



### Paso 2.1: Plugin "Hola Mundo"

#### PROMPT A LA IA:

Dame código completo para plugin QGIS "catastro\_loader" con menú "Catastro", item "Cargar parcelas" que muestre un QMessageBox con "Plugin funcionando". Incluye \_\_init\_\_.py, catastro\_loader.py, metadata.txt, resources.py.

### Paso 2.2: Integrar lógica de descarga

#### PROMPT A LA IA:

Integra el script test\_parcelas.py en el plugin. Usa QDialog para municipio, QProgressDialog para progreso, QMessageBox para resultado. Ejecuta en QThread para no congelar QGIS.

## Paso 2.3: Cargar GML en QGIS



### PROMPT A LA IA:

Añade función que localice el .gml, cargue como QgsVectorLayer, añada al proyecto, haga zoom, maneje errores y opcionalmente elimine ZIP temporal.



## Fase 3: Interfaz de Usuario Completa



### Paso 3.1: Diálogo con Qt



### PROMPT A LA IA:

Crea un QDialog con:  
QComboBox (Parcelas/Direcciones/Edificios), QLineEdit (municipio), botón Examinar para carpeta, QCheckBox "Eliminar ZIP", botones Descargar/Cancelar, QProgressBar. Guarda carpeta con QSettings.

### Paso 3.2: Conectar los 3 ATOMs



### PROMPT A LA IA:

Modifica para usar ATOM según selección: Parcelas (CP), Direcciones (AD), Edificios (BU). Refactoriza lógica común en función `descargar_datos(tipo, municipio, carpeta)`.

### Paso 3.3: Autocompletado



#### PROMPT A LA IA:

Añade QCompleter al campo municipio. Al escribir 3 caracteres, buscar del ATOM actual. Cachear lista para no recargar.

## Fase 4: Robustez y Calidad



### Paso 4.1: Manejo de errores



#### PROMPT A LA IA:

Añade manejo de errores para: sin internet, municipio no existe, servidor caído, ZIP corrupto, GML mal formado, cancelación usuario, espacio en disco, permisos. Mensajes en español y log a archivo.

## Paso 4.2: Refactorizar



### PROMPT A LA IA:

Separa en módulos:  
downloader.py, parser.py, dialog.py, utils.py. Cada  
función una responsabilidad, type hints, docstrings  
Google, máximo 50 líneas.

## Paso 4.3: Documentar



### PROMPT A LA IA:

Genera README.md, INSTALL.md,  
CHANGELOG.md, docs/user\_guide.md,  
docs/developer\_guide.md, docs/glossary.md. Todo en  
Markdown.



## Fase 5: Pruebas y Publicación



### Paso 5.1: Script de pruebas



### PROMPT A LA IA:

Crea test\_plugin.py que pruebe conexión a ATOMs, búsqueda de municipio, descarga de municipio pequeño ("Ábalos"), valide GML, genere informe HTML. Sin QGIS.

## Paso 5.2: Publicar en GitHub



### PROMPT A LA IA:

Dame comandos git para inicializar, hacer commit, conectar con GitHub remoto. Genera .gitignore para Python/QGIS. Dame texto descripción y 5 tags.

## Paso 5.3: Repositorio oficial QGIS



### PROMPT A LA IA:

Prepara plugin para plugins.qgis.org: estructura carpetas, metadata.txt completo con todos campos obligatorios, cómo crear ZIP, lista de requisitos.



## Fase 6: Aprendizaje y Mantenimiento



### Paso 6.1: Diario diario



#### PROMPT A LA IA:

Resume la sesión de hoy en formato: FECHA, TIEMPO, OBJETIVO, LOGROS, ERRORES, 3 COSAS APRENDIDAS, PROMPT QUE ME FUNCIONÓ MEJOR. Guarda en docs/diario\_[fecha].md

### Paso 6.2: Sistema de logs



#### PROMPT A LA IA:

Añade logging con archivo por día, niveles DEBUG/INFO/WARNING/ERROR, registro automático de peticiones HTTP y capas cargadas. Función `exportar_logs_soporte()` a ZIP.



## Checklist Final

- ☐ **Funcionamiento básico**
- ☐ ☐ El plugin aparece en el menú "Plugins" de QGIS

- ☐ ☐ Al hacer clic, se abre el diálogo
- ☐ ☐ Se pueden seleccionar Parcelas, Direcciones y Edificios
- ☐ ☐ El autocompletado de municipios funciona
- ☐ ☐ La descarga se completa sin errores
- ☐ ☐ La capa se carga automáticamente en QGIS
- ☐ **Manejo de errores**
- ☐ ☐ Sin internet: muestra mensaje amigable
- ☐ ☐ Municipio no existe: muestra sugerencias
- ☐ ☐ Cancelar descarga: no deja archivos basura
- ☐ ☐ Servidor caído: reintenta y avisa
- ☐ **Documentación**
- ☐ ☐ README.md existe y está completo
- ☐ ☐ Comentarios en código claros
- ☐ ☐ Instrucciones de instalación
- ☐ **Calidad de código**
- ☐ ☐ Código separado en módulos
- ☐ ☐ Funciones con docstrings
- ☐ ☐ Sin variables sin uso



## Anexo: Solución Rápida para Errores Frecuentes

Error: "No module named requests"



```
pip install requests
```

```
C:\ruta\QGIS\bin\python.exe -m pip install requests
```

## Error: "No provider for layer" al cargar GML



### PROMPT A LA IA:

Dame un código de ejemplo que convierta un GML a GeoPackage usando PyQGIS. Así puedo cargar el GeoPackage en lugar del GML directamente.

## Error: Plugin no aparece en QGIS



- Verifica que la carpeta se llama exactamente igual que el plugin
- Verifica metadata.txt tiene formato correcto
- Reinicia QGIS completamente
- Plugins → Administrar → Configuración → Ver logs



## Comandos Útiles

```
# Recargar plugin sin reiniciar QGIS
import importlib
if 'catastro_loader' in sys.modules:
    importlib.reload(sys.modules['catastro_loader'])

# Ver carpeta de plugins
from qgis.core import QgsApplication
print(QgsApplication.qgisSettingsDirPath() + "python/plugins")

# Ver drivers GDAL
from osgeo import gdal
```

```
for i in range(gdal.GetDriverCount()):  
    print(gdal.GetDriver(i).ShortName)
```

 Plan creado para tu primer plugin QGIS |  Usa Twinny + DeepSeek |  Ve paso a paso, sin prisa

☒ Cada prompt copia y pega en la IA | ☐ Marca tus avances